

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I ARUNPRASATH R 192524077 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ARUNPRASATH R

Q. No	Conceptual Understanding (20)	Model Design (25)	Tool Usage(20)	Analysis of Results(20)	Documentation(15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Annexure A

SIMULATION TASK

.Q1. A compiler receives an intermediate representation of an arithmetic expression containing multiple operators and temporary values.

Simulation Tasks

- i. Simulate instruction selection for the arithmetic operations.
- ii. Allocate registers for operands and temporary values.
- iii. Generate the corresponding target instruction sequence.
- iv. Show the register contents after each instruction.
- v. Evaluate the efficiency of the generated code. (10 Marks)

ANS:

Goal

Start with an intermediate code such as:

$t1 = b + c$

$t2 = t1 * d$

$a = t2$

Then simulate how a compiler converts it into target instructions.

Step 1 — Create Python file

Create:

q1.py

Step 2 — Put this code

Q1: Instruction Selection and Register Allocation

```
print("INTERMEDIATE CODE")
```

```
print("t1 = b + c")
```

```
print("t2 = t1 * d")
```

```
print("a = t2")
```

```
print("\nTARGET CODE")
```

```
print("MOV R1, b")
```

```
print("ADD R1, c")
```

```
print("MOV R2, R1")
```

```
print("MUL R2, d")
```

```
print("MOV a, R2")
```

```
print("\nREGISTER CONTENTS")
```

```
print("After MOV R1,b -> R1 = b")
```

```
print("After ADD R1,c -> R1 = b+c")
```

```
print("After MOV R2,R1 -> R2 = b+c")
```

```
print("After MUL R2,d -> R2 = (b+c)*d")
```

```
print("After MOV a,R2 -> a = (b+c)*d")
```

Step 3 — Run

In CMD:

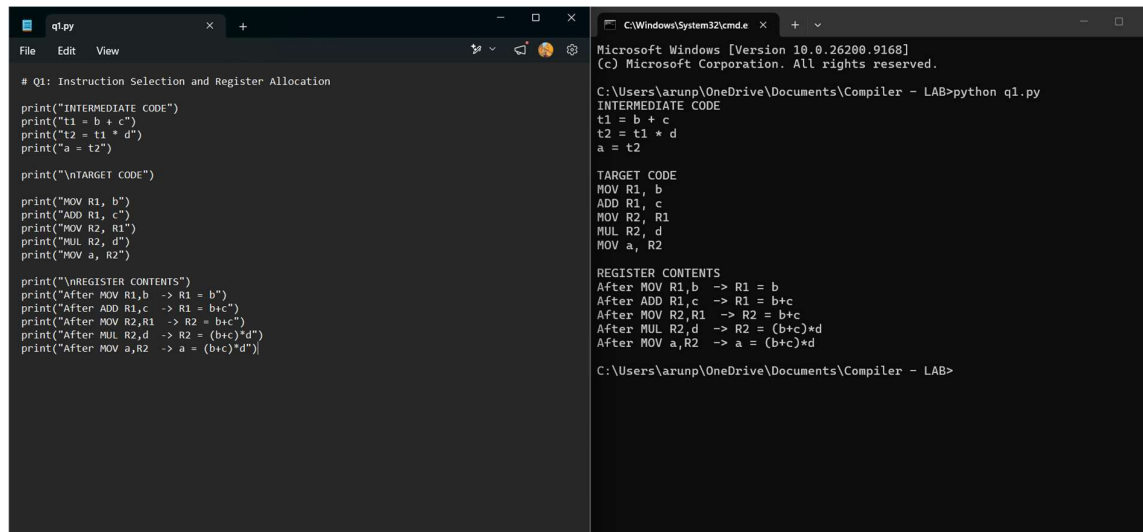
```
python q1.py
```

Step 4 — Explain the output

You can tell faculty:

First, the intermediate representation is given. Each arithmetic operation is converted into target instructions. Registers R1 and R2 are allocated for temporary values. After every instruction, the register contents are displayed.

This directly demonstrates the tasks of instruction selection, register allocation, target instruction generation, and register tracking.



The screenshot shows a Python script named `q1.py` in a text editor and its execution output in a Windows command prompt. The script is titled "# Q1: Instruction Selection and Register Allocation" and contains several print statements to show intermediate and target code, as well as register contents after each instruction.

```
# Q1: Instruction Selection and Register Allocation

print("INTERMEDIATE CODE")
print("t1 = b + c")
print("t2 = t1 * d")
print("a = t2")

print("\nTARGET CODE")

print("MOV R1, b")
print("ADD R1, c")
print("MOV R2, R1")
print("MUL R2, d")
print("MOV a, R2")

print("\nREGISTER CONTENTS")
print("After MOV R1,b -> R1 = b")
print("After ADD R1,c -> R1 = b+c")
print("After MOV R2,R1 -> R2 = b+c")
print("After MUL R2,d -> R2 = (b+c)*d")
print("After MOV a,R2 -> a = (b+c)*d")
```

The command prompt output shows the execution of the script, displaying the intermediate code, target code, and register contents as specified in the script.

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>python q1.py

INTERMEDIATE CODE
t1 = b + c
t2 = t1 * d
a = t2

TARGET CODE
MOV R1, b
ADD R1, c
MOV R2, R1
MUL R2, d
MOV a, R2

REGISTER CONTENTS
After MOV R1,b -> R1 = b
After ADD R1,c -> R1 = b+c
After MOV R2,R1 -> R2 = b+c
After MUL R2,d -> R2 = (b+c)*d
After MOV a,R2 -> a = (b+c)*d

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>
```

Q2.A sequence of nested procedure invocations is represented in intermediate form.

Simulation Tasks

- Simulate runtime stack creation.
- Construct activation records.
- Allocate storage for parameters and local variables.
- Show stack changes during procedure calls and returns.
- Explain the role of runtime storage management during code generation. (10 Marks)

ANS:

Suppose:

main()

↓

function A()

↓

function B()

Step 1

Create:

q2.py

Step 2

Use:

Q2: Runtime Stack Simulation

```
stack = []
```

```
def call_function(name, parameters, locals):
```

```
    record = {
```

```
        "function": name,
```

```
        "parameters": parameters,
```

```
        "locals": locals
```

```
    }
```

```
    stack.append(record)
```

```
    print("\nCALL:", name)
```

```
    print("Stack:")
```

```
    for item in stack:
```

```
        print(item)
```

```
def return_function():
```

```
    record = stack.pop()
```

```
    print("\nRETURN:", record["function"])
```

```
    print("Stack:")
```

```
    for item in stack:
```

```
        print(item)
```

```
print("RUNTIME STACK SIMULATION")
```

```
call_function("main", [], ["x", "y"])
```

```
call_function("A", ["10"], ["a"])
```

```
call_function("B", ["20"], ["b"])
```

return_function()

return_function()

return_function()

Step 3

Run:

python q2.py

What happens?

Initially:

main

Then:

main

A

Then:

main

A

B

When B returns:

main

A

When A returns:

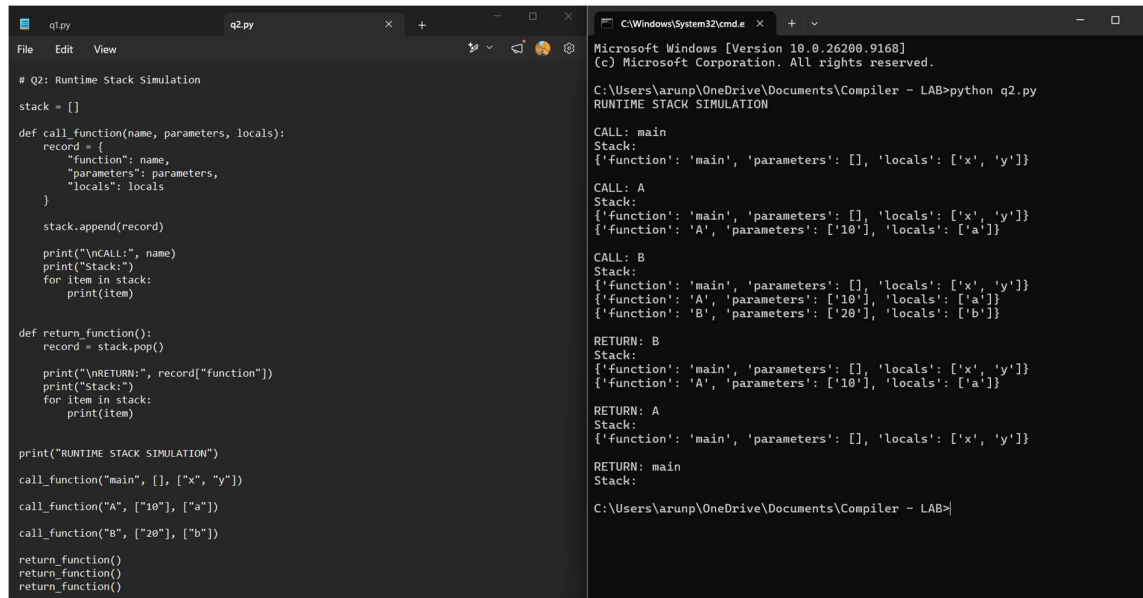
main

Finally:

empty stack

Each function call creates an activation record and pushes it onto the runtime stack. The activation record stores parameters and local variables. When the function returns, its activation record is removed from the stack.

This covers runtime stack creation, activation records, storage allocation, stack changes during calls/returns, and runtime storage management



The image shows a screenshot of a Python script and its execution output. The script, named `q2.py`, simulates a runtime stack. It defines a `call_function` function that pushes activation records onto a stack and a `return_function` function that pops them. The script then calls `main`, `A`, `B`, and returns. The output on the right shows the state of the stack at each step.

```
# Q2: Runtime Stack Simulation
stack = []

def call_function(name, parameters, locals):
    record = {
        "function": name,
        "parameters": parameters,
        "locals": locals
    }
    stack.append(record)

    print("\nCALL:", name)
    print("Stack:")
    for item in stack:
        print(item)

def return_function():
    record = stack.pop()

    print("\nRETURN:", record["function"])
    print("Stack:")
    for item in stack:
        print(item)

print("RUNTIME STACK SIMULATION")
call_function("main", [], ["x", "y"])
call_function("A", ["10"], ["a"])
call_function("B", ["20"], ["b"])

return_function()
return_function()
return_function()
```

Microsoft Windows [Version 10.0.26280.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>python q2.py

RUNTIME STACK SIMULATION

CALL: main
Stack:
{'function': 'main', 'parameters': [], 'locals': ['x', 'y']}

CALL: A
Stack:
{'function': 'main', 'parameters': [], 'locals': ['x', 'y']}
{'function': 'A', 'parameters': ['10'], 'locals': ['a']}

CALL: B
Stack:
{'function': 'main', 'parameters': [], 'locals': ['x', 'y']}
{'function': 'A', 'parameters': ['10'], 'locals': ['a']}
{'function': 'B', 'parameters': ['20'], 'locals': ['b']}

RETURN: B
Stack:
{'function': 'main', 'parameters': [], 'locals': ['x', 'y']}
{'function': 'A', 'parameters': ['10'], 'locals': ['a']}

RETURN: A
Stack:
{'function': 'main', 'parameters': [], 'locals': ['x', 'y']}

RETURN: main
Stack:

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>

Q3. Next-use information for variables is available before code generation begins.

Simulation Tasks

- Construct the next-use table.
- Simulate register allocation based on next-use information.
- Release registers whenever values become inactive.
- Generate the final target instruction sequence.
- Explain how next-use information improves register utilization. (10 Marks)

ANS:

Use this intermediate code:

1. $a = b + c$

2. $d = a + e$

3. $f = d + b$

We determine **when each variable will be used next**.

Step 1

Create:

q3.py

Step 2

Q3: Next-Use Information

```
instructions = [  
    "a = b + c",  
    "d = a + e",  
    "f = d + b"  
]  
  
print("INTERMEDIATE CODE")  
  
for i, instruction in enumerate(instructions, 1):  
    print(i, instruction)  
  
print("\nNEXT-USE TABLE")  
  
print("Statement 1: a = b + c")  
print("b -> next use: Statement 3")  
print("c -> no next use")  
  
print("a -> next use: Statement 2")  
print("\nStatement 2: d = a + e")  
print("a -> used here")  
print("e -> no next use")  
  
print("d -> next use: Statement 3")  
print("\nStatement 3: f = d + b")  
print("d -> used here")  
print("b -> used here")  
print("f -> no next use")  
  
print("\nREGISTER ALLOCATION")
```



```

print("R1 <- b")

print("R2 <- c")

print("R1 = b + c")

print("R2 <- a")

print("R2 = a + e")

print("R2 <- d")

print("R2 = d + b")

print("\nRegisters are released when variables have no further use.")

```

```

q1.py q2.py q3.py
File Edit View
instructions = [
    "a = b + c",
    "d = a + e",
    "f = d + b"
]

print("INTERMEDIATE CODE")
for i, instruction in enumerate(instructions, 1):
    print(i, instruction)

print("\nNEXT-USE TABLE")

print("Statement 1: a = b + c")
print("b -> next use: Statement 3")
print("c -> no next use")
print("a -> next use: Statement 2")

print("\nStatement 2: d = a + e")
print("a -> used here")
print("e -> no next use")
print("d -> next use: Statement 3")

print("\nStatement 3: f = d + b")
print("d -> used here")
print("b -> used here")
print("f -> no next use")

print("\nREGISTER ALLOCATION")

print("R1 <- b")
print("R2 <- c")
print("R1 = b + c")
print("R2 <- a")
print("R2 = a + e")
print("R2 <- d")
print("R2 = d + b")

print("\nRegisters are released when variables have no further use.")pytho

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26280.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>python q3.py
INTERMEDIATE CODE
1 a = b + c
2 d = a + e
3 f = d + b

NEXT-USE TABLE
Statement 1: a = b + c
b -> next use: Statement 3
c -> no next use
a -> next use: Statement 2

Statement 2: d = a + e
a -> used here
e -> no next use
d -> next use: Statement 3

Statement 3: f = d + b
d -> used here
b -> used here
f -> no next use

REGISTER ALLOCATION
R1 <- b
R2 <- c
R1 = b + c
R2 <- a
R2 = a + e
R2 <- d
R2 = d + b

Registers are released when variables have no further use.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>

```

Q4. An intermediate instruction sequence contains repeated computations inside a loop.

Simulation Tasks

- Identify computations repeated during loop execution.
- Move loop-invariant computations outside the loop.
- Generate the optimized loop.
- Compare the execution cost before and after optimization.
- Explain why loop optimization improves performance. (10 Marks)

ANS:

Consider:

```
for(i = 0; i < n; i++)
```

```
{  
    x = a * b;  
    y = x + i;  
}
```

Here:

```
x = a * b
```

doesn't depend on i.

So calculating it every iteration is unnecessary.

Before optimization

```
for(i = 0; i < n; i++)
```

```
{  
    x = a * b;  
    y = x + i;  
}
```

After optimization

```
x = a * b;
```

```
for(i = 0; i < n; i++)
```

```
{  
    y = x + i;  
}
```

Python implementation

Create:

q4.py

Q4: Loop-Invariant Code Motion

```
print("BEFORE OPTIMIZATION")
```

```
print("""
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    x = a * b;
```

```
    y = x + i;
```

```
}
```

```
""")
```

```
print("Loop-invariant computation:")
```

```
print("x = a * b")
```

```
print("\nAFTER OPTIMIZATION")
```

```
print("""
```

```
x = a * b;
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    y = x + i;
```

```
}
```

```
""")
```

```
print("\nEXECUTION COST")
```

```
print("Before: a*b is calculated n times")
```

```
print("After : a*b is calculated only once")
```

```
print("\nOptimization reduces repeated computation.")
```

The screenshot shows a code editor with a file named q4.py and a Windows command prompt window. The code in q4.py is a Python script demonstrating loop-invariant code motion optimization. It prints 'BEFORE OPTIMIZATION', shows a loop where 'x = a * b' is calculated repeatedly, prints 'AFTER OPTIMIZATION' where 'x = a * b' is moved outside the loop, and prints 'EXECUTION COST' comparing the number of calculations before and after optimization. The command prompt shows the execution of 'python q4.py', displaying the same output as the code's print statements.

```
# Q4: Loop-Invariant Code Motion
print("BEFORE OPTIMIZATION")

print("""
for(i = 0; i < n; i++)
{
    x = a * b;
    y = x + i;
}
""")

print("Loop-invariant computation:")
print("x = a * b")

print("\nAFTER OPTIMIZATION")

print("""
x = a * b;
for(i = 0; i < n; i++)
{
    y = x + i;
}
""")

print("\nEXECUTION COST")

print("Before: a*b is calculated n times")
print("After : a*b is calculated only once")

print("\nOptimization reduces repeated computation.")
```

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>python q4.py
BEFORE OPTIMIZATION
for(i = 0; i < n; i++)
{
 x = a * b;
 y = x + i;
}
Loop-invariant computation:
x = a * b
AFTER OPTIMIZATION
x = a * b;
for(i = 0; i < n; i++)
{
 y = x + i;
}
EXECUTION COST
Before: a*b is calculated n times
After : a*b is calculated only once
Optimization reduces repeated computation.
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>

Q5. A basic block contains multiple repeated arithmetic expressions.

Simulation Tasks

- Construct the Directed Acyclic Graph (DAG).
- Identify common subexpressions.
- Eliminate redundant computations.
- Generate optimized intermediate instructions from the DAG.
- Compare the original and optimized instruction sequences. (10 Marks)

ANS:

This is probably the **most important one to understand** because your document specifically asks for a Directed Acyclic Graph.

Take:

$t1 = a + b$

$t2 = a + b$

$t3 = t1 * c$

$t4 = t2 * c$

Notice:

$a + b$

is calculated twice.

And:

$t1 * c$

$t2 * c$

represent the same computation because $t1$ and $t2$ contain the same value.

Before optimization

$t1 = a + b$

$t2 = a + b$

$t3 = t1 * c$

$t4 = t2 * c$

DAG idea



The important point is that **one $a+b$ node is shared by both $t1$ and $t2$.**

Python implementation

Create:

q5.py

Q5: DAG Based Common Subexpression Elimination

```
print("ORIGINAL CODE")

print("t1 = a + b")

print("t2 = a + b")

print("t3 = t1 * c")

print("t4 = t2 * c")

print("\nCOMMON SUBEXPRESSION")

print("a + b occurs more than once.")

print("\nDAG REPRESENTATION")

print("""
    +
  /\
 a  b
 |
t1,t2
 |
 *
 /\
t1  c
 |
t3,t4
""")

print("OPTIMIZED CODE")

print("t1 = a + b")
```

```
print("t3 = t1 * c")
```

```
print("t4 = t1 * c")
```

```
print("\nRESULT")
```

```
print("Repeated computation a+b is eliminated.")
```

```
print("The DAG identifies and shares the common subexpression.")
```

The screenshot shows a code editor with a Python script and a terminal window displaying its output.

Python Script (q5.py):

```
# Q5: DAG Based Common Subexpression Elimination

print("ORIGINAL CODE")

print("t1 = a + b")
print("t2 = a + b")
print("t3 = t1 * c")
print("t4 = t2 * c")

print("\nCOMMON SUBEXPRESSION")

print("a + b occurs more than once.")

print("\nDAG REPRESENTATION")

print("""
      +
     /\
    a  b
     |
    t1,t2
     |
     +
    /\
   t1  c
     |
    t3,t4
""")

print("OPTIMIZED CODE")

print("t1 = a + b")
print("t3 = t1 * c")
print("t4 = t1 * c")

print("\nRESULT")

print("Repeated computation a+b is eliminated.")
print("The DAG identifies and shares the common subexpression.")
```

Terminal Output:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>python q5.py

ORIGINAL CODE
t1 = a + b
t2 = a + b
t3 = t1 * c
t4 = t2 * c

COMMON SUBEXPRESSION
a + b occurs more than once.

DAG REPRESENTATION

      +
     /\
    a  b
     |
    t1,t2
     |
     +
    /\
   t1  c
     |
    t3,t4

OPTIMIZED CODE
t1 = a + b
t3 = t1 * c
t4 = t1 * c

RESULT
Repeated computation a+b is eliminated.
The DAG identifies and shares the common subexpression.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>
```

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation